

לימוד המאגר הזה מהיסודות

מערכת מבוזרת לעיבוד משימות רקע (Distributed Background Job Processing)
FastAPI + PostgreSQL + Redis Streams + Docker Compose

נכתב עבור קורא/ת שלא בנו את המערכת, בעלי ניסיון עמוק ב-Palantir Foundry, ושמעולם לא עבדו עם בסיסי נתונים לעומק, Redis, מתווכי הודעות (Brokers) או מערכות מקביליות/מבוזרות. כל מושג מוסבר לפני השימוש בו, וממופה ל-Foundry בכל מקום שבו קיימת הקבלה נאמנה.

תוכן העניינים

1. מהי המערכת הזו, ולמה היא קיימת
2. רקע: הטכנולוגיות, מאפס
3. המערכת במבט-על
4. חייה של משימה: המסלול התקין, מקצה לקצה
5. מודל הנתונים: טבלה אחת שמספרת את כל הסיפור
6. צד ה-Redis: Streams, Consumer Groups וקבוצת ההשהיה
7. הבעיות הקשות, אחת-אחת
8. סיור מודרך בקוד
9. בדיקות, הרצה ותצפית (Observability)
10. מגבלות, פשרות ומילון מונחים

איך לקרוא: חלקים 1-3 הם רקע והתמצאות (קריאה חד-פעמית). חלק 4 הוא הסיור המרכזי — כדאי לקרוא אותו עם הקוד פתוח. חלקים 5-7 הם לב העניין. חלקים 8-10 הם חומר עזר. תיבות "מקבילה ל-Foundry" ממפות כל רעיון לפלטפורמה שאתם כבר מכירים; הן עזרי הבנה, לא הקבלות מדויקות.

1. מהי המערכת הזו, ולמה היא קיימת

המאגר הזה מממש מערכת מבוזרת לעיבוד משימות רקע. לקוח שולח "משימה" (job) — יחידת עבודה כמו "שלח את האימייל הזה" או "הפק את הדוח הזה" — דרך HTTP API. ה-API **לא** מבצע את העבודה. הוא רושם את המשימה, עונה "התקבל" תוך אלפיות שנייה, והעבודה עצמה מתבצעת מאוחר יותר, בתהליך (process) נפרד, אולי על מכונה אחרת. הלקוח יכול לתשאל סטטוס, לראות את התוצאה, לבטל את המשימה, או להריץ אותה מחדש אם נכשלה.

מקבילה ל-Foundry

אתם משתמשים בדיוק בדפוס הזה כל יום. כשאתם שומרים Transform ולוחצים "Build" ב-Foundry, הדפדפן שלכם לא מריץ Spark. שירות ה-Build רושם בקשת build, שם אותה בתור (queue) ועונה מיד — דף ה-build מציג QUEUED. כעבור זמן-מה מודול Spark אוסף את ה-build, מריץ אותו, והסטטוס עובר ל-RUNNING ואז ל-SUCCEEDED או FAILED. המאגר הזה הוא גרסה מיניאטורית ועצמאית של אותה מכונה: ה-API הוא שירות ה-Build, משימות ה-builds, ה-workers הם מודולי Spark, וטבלת המשימות היא דף היסטוריית ה-builds.

למה לא פשוט לבצע את העבודה בתוך בקשת ה-HTTP?

- **איטיות:** משימת דוח לוקחת 3–5 שניות. החזקת חיבור HTTP פתוח כל-כך הרבה זמן מבזבזת קיבולת שרת ונופלת בקלות על timeout בעומס.
- **אמינות:** אם השרת מופעל מחדש באמצע בקשה, העבודה אובדת בשקט. עם תור, המשימה הרשומה שורדת ונאספת שוב.
- **סקיילינג:** תעבורת web ועבודת רקע גדלות בקצבים שונים. הפרדה ביניהן מאפשרת להוסיף workers בלי לגעת ב-API.
- **ניסיונות חוזרים (Retries):** כשלים חולפים (webhook חיצוני רעוע) צריכים לקבל retry אוטומטי כעבור דקות. בקשת HTTP לא יכולה לחכות דקות.

מקבילה ל-Foundry

מאותה סיבה בדיוק builds ב-Foundry הם אסינכרוניים: pipeline יכול לרוץ שעות, מכונות יכולות להיעצר באמצע, ו-Foundry מריץ מחדש builds רעועים — שום דבר מזה לא היה אפשרי אם בקשת הדפדפן עצמה הייתה מבצעת את החישוב.

ארבעת סוגי המשימות המדומים

- מכיוון שזהו פרויקט מבחן, המשימות מדמות עבודה במקום לבצע עבודה אמיתית. לכל סוג יש handler ב-app/jobs/handlers.py:
- **email** — ישן 1–3 שניות, מחזיר מזהה הודעה מדומה.
 - **webhook** — ישן 1–2 שניות, ואז נכשל אקראית ב-20% מהמקרים. הוא קיים במיוחד כדי להפעיל את מנגנון ה-retry.
 - **report** — ישן 3–5 שניות, מחזיר URL מדומה של קובץ.
 - **batch** — מעבד רשימת פריטים אחד-אחד, מעדכן אחוז התקדמות תוך כדי, ומחזיר סיכום. זהו סוג המשימה היחיד שניתן לבטל באמצע ריצה.

2. רקע: הטכנולוגיות, מאפס

החלק הזה מסביר את שלוש הטכנולוגיות שהמאגר בנוי עליהן — PostgreSQL, Redis, והרעיון של תור הודעות — בהנחה של אפס חשיפה מוקדמת. אם אתם מכירים אותן, דלגו לחלק 3.

2.1 PostgreSQL: בסיס הנתונים הרלציוני

בסיס נתונים רלציוני מאחסן נתונים בטבלאות: אוספים בעלי שם של שורות, כשלכל שורה אותן עמודות מוגדרות-טיפוס — בדומה לגיליון אלקטרוני עם סכימה כפוייה. PostgreSQL ("פוסטגרס") הוא אחד מבסיסי הנתונים הרלציוניים בקוד פתוח הנפוצים בעולם. מדברים איתו ב-SQL, שפה הצהרתית: "הכנס שורה" (INSERT), "שלוף שורות שבהן status=pending" (SELECT), "עדכן שורה זו ל-status=processing" (UPDATE).

מקבילה ל-Foundry

טבלת פוסטגרס היא כמו dataset ב-Foundry עם סכימה קשיחה — אבל מתוכננת למיליוני קריאות וכתובות זעירות ומיידיות של שורות בודדות, במקום טרנספורמציות bulk של קבצים שלמים. איפה ש-pipeline ב-Foundry משכתב dataset בטרנזקציה גדולה אחת לכל build, פוסטגרס מטפל באלפי שינויים ברמת-שורה בשנייה, כל אחד מהם נראה מיידית.

שלושה מושגי פוסטגרס קריטיים להבנת המאגר:

- **טרנזקציות (TRANSACTIONS):** קבוצת שינויים שכולם קורים או שאף אחד לא קורה ("אטומיות"). אם התהליך מת באמצע טרנזקציה, בסיס הנתונים מתנהג כאילו היא מעולם לא התחילה. מסיימים טרנזקציה ב-COMMIT (קיבוע) או ROLLBACK (ביטול).
- **עדכונים מותנים אטומיים:** פקודה כמו `UPDATE jobs SET status='processing' WHERE id=X AND status='pending'` בודקת את התנאי ומחילה את השינוי כפעולה אחת בלתי-ניתנת-לחלוקה. שני תהליכים שמתחרים להריץ אותה לא יכולים להצליח שניהם: בסיס הנתונים מסדר אותם בטור, אחד מתאים לתנאי ה-WHERE ומעדכן את השורה, והשני לא מוצא שורה מתאימה ולא משנה כלום. **התכונה הבודדת הזו היא היסוד לכל הנכונות (correctness) של המאגר — זכרו אותה.**
- **אינדקסים (INDEXES):** מבני חיפוש ממוינים על עמודות נבחרות, כך ש"מצא את כל השורות עם status=pending" סורק אינדקס קטן במקום את כל הטבלה. טבלת ה-jobs מאונדקסת על status, על priority ועל (status, created_at).

מקבילה ל-Foundry

טרנזקציות של datasets ב-Foundry נותנות את אותו רעיון "הכל-או-כלום" בקנה מידה של קבצים: build שנכשל לעולם לא משאיר dataset חצי-כתוב. פוסטגרס נותן את אותה הבטחה בקנה מידה של שורות בודדות, אלפי פעמים בשנייה.

Alembic ו-SQLAlchemy 2.2

קוד הפיתוח לעולם לא כותב מחרוזות SQL גולמיות ביד. SQLAlchemy היא ספרייה שממפה מחלקות פיתוח לטבלאות — מחלקת Job ב-app/models/job.py היא הגדרת טבלת ה-jobs — ומחוללת SQL בטוח ומפורמט-פרמטרים. Alembic הוא כלי המיגרציות הנלווה: כל שינוי סכימה הוא סקריפט ממוספר-גרסה, alembic-head upgrade מריץ אותם ומביא כל בסיס נתונים ריק לסכימה הנוכחית. ב-docker-compose, קונטיינר חד-פעמי בשם migrate מריץ זאת לפני שכל השאר עולים.

מקבילה ל-Foundry

מיגרציות Alembic הן לבסיס הנתונים מה שקוד Transform ב-commit הוא ל-pipeline ב-Foundry: הסכימה מוגדרת בקוד מנוהל-גרסאות ומוחלת באופן שחזיר (reproducible), לעולם לא "מוקלקת" ידנית.

Redis 2.3: שרת הנתונים בזיכרון

Redis הוא שרת שמחזיק מבני נתונים ב-RAM וחושף פקודות לתפעולם דרך הרשת. מכיוון שהכל חי בזיכרון, פעולות לוקחות מיקרו-שניות. הוא לא בסיס נתונים כללי — חשבו עליו כעל לוח טיוטה משותף ומהיר מאוד, שתהליכים רבים על מכונות רבות קוראים וכותבים אליו בו-זמנית. Redis מציע כמה סוגי מבנים; המאגר הזה משתמש בשניים:

- **STREAM**: יומן הודעות בהוספה-בלבד (append-only), שכל רשומה בו מקבלת מזהה אוטומטי עולה. יצרנים מוסיפים עם XADD; צרכנים קוראים רשומות. המאגר משתמש בשלושה streams כתורי המשימות שלו (אחד לכל עדיפות).
- **SORTED SET (ZSET)**: קבוצת מחרוזות שלכל אחת ניקוד מספרי, שמורה ממוינת לפי הניקוד תמידית. שאלו "תן לי את כל האיברים עם ניקוד $\geq$ עכשיו" ותקבלו אותם מיידיית. המאגר מאחסן שם מזהי משימות מתוזמנות-עתידית, עם חותמת-הזמן היעודה כניקוד.

עוד תכונה אחת של Redis חשובה כאן: פקודות בודדות הן אטומיות (השרת מבצע פקודה אחת בכל רגע), וקובץ ה-compose מדליק התמדה מסוג append-only-file כך ש-Redis שורד הפעלות מחדש. ובכל זאת, העיצוב מתייחס ל-Redis כאל בר-החלפה: פוסטגרס הוא תמיד מקור האמת, וחלק 6 מראה איך המערכת בונה מחדש את מצב Redis מפוסטגרס אם הוא אבד.

2.4 מהו תור הודעות / broker (ואיפה RabbitMQ נכנס)

תור הודעות הוא חוצץ (buffer) בין יצרן ("צריך לבצע את המשימה הזו") לצרכנים ("אני מבצע משימות"). היצרן מוסיף הודעה וממשיך הלאה; הצרכנים מושכים הודעות כשיש להם קיבולת. זה מנתק את שני הצדדים בזמן (היצרן לא מחכה), במרחב (מכונות שונות) ובקצב (פרץ של 10,000 הגשות רק מאריך את התור; ה-workers מרוקנים אותו בקצב שלהם).

RabbitMQ, Kafka ו-AWS SQS הם מוצרי "broker" ייעודיים שזו כל מהותם. המאגר הזה בונה תור מ-Redis Streams במקום — בחירה נפוצה ולגיטימית לפרודקשן כשכבר מריצים Redis והצרכים צנועים. המושגים זהים; רק המוצר שונה.

החלק הקשה בכל תור הוא סמנטיקת המסירה (delivery semantics). אם צרכן לוקח הודעה ואז קורס — ההודעה אבדה? קיימים שלושה חוזים:

- **At-most-once**: ההודעה נמחקת ברגע המסירה. קריסה מאבדת אותה. מהיר אך מאבד נתונים.
- **At-least-once**: ההודעה נשארת "מושאלת" עד שהצרכן מאשר (acknowledge/ack) במפורש שסיים. קריסה = ההודעה תימסר מחדש למישהו אחר בהמשך. שום דבר לא אובד, אבל אותה הודעה עשויה להיות מעובדת פעמיים — ולכן העיבוד חייב לסבול כפילויות.
- **Exactly-once**: מונח שיווקי. במערכות מבוזרות אמיתיות משיגים אותו על-ידי at-least-once **ועוד** סינון-כפילויות בצד הצרכן — וזה בדיוק מה שהמאגר הזה עושה (חלק 7.1).

מקבילה ל-FOUNDRY

תור ה-builds של Foundry מבטיח אותו דבר: אם המודול שמריץ את ה-build שלכם נעצר או מת, ה-build לא מתאדה — הוא חוזר לתור ורץ שוב. ראינו builds מתחילים מחדש אחרי שיהוק תשתיתי; זהו at-least-once delivery, כפי שהוא נחוץ מצד המשתמש.

2.5 מקבילות ומערכות מבוזרות: שתי העובדות שמייצרות כל בעיה

המאגר מריץ תהליכים עצמאיים רבים: API אחד (שבעצמו מטפל בבקשות רבות במקביל), workers N, ו-ticker אחד. הם לא חולקים שום זיכרון — הם יכולים לתאם רק דרך פוסטגרס ו-Redis. נובעות שתי עובדות, וכל בעיה קשה בחלק 7 היא תוצאה של אחת מהן או שתיהן:

- **עובדה 1 — שזירה (Interleaving):** בין כל שתי שורות בקוד שלכם, תהליך אחר עשוי להריץ כל מספר של שורות משלו. קוד שקורא ערך, חושב עליו, ואז כותב ("check-then-act") שבור כברירת מחדל: הערך אולי השתנה בין הקריאה לכתיבה. התיקון: להפוך את הבדיקה והפעולה לפעולה אטומית אחת — בדיוק מה שהעדכון המותנה האטומי מסעיף 2.1 מספק.
 - **עובדה 2 — כשל חלקי (Partial failure):** כל תהליך יכול למות בין כל שתי שורות, והאחרים לא יכולים לדעת זאת ישירות. worker שהשתתק אולי מת, אולי איטי, אולי הרשת מפילה את החבילות שלו — שלושת המצבים נראים זהים מבחוץ. מערכות מתמודדות עם זה בעזרת timeouts ("שתק יותר מדי זמן = מוחזק כמת") ובכך שהופכות את פעולות ההתאוששות לבטוחות גם אם ההנחה הייתה שגויה.
- דוגמה קונקרטית ל-race condition, כי זה מפיל את כולם בפעם הראשונה. שני workers רוצים את משימה J. קוד נאיבי: (1) קרא את הסטטוס של J — שניהם רואים 'pending'; (2) שניהם מסיקים "היא פנויה"; (3) שניהם קובעים 'status='processing' ושניהם מריצים את המשימה. האימייל נשלח פעמיים. הפער בין הקריאה לכתיבה הוא הבאג. התיקון של המאגר: לדלג על הקריאה לגמרי ולהוציא `UPDATE ... SET status='processing' WHERE id=J` — `AND status='pending'` בסיס הנתונים מבטיח שבדיוק worker אחד ימצא התאמה, וכל אחד לומד ממספר-השורות (0 או 1) אם ניצח.

מקבילה ל-FOUNDRY

Foundry מגן עליכם מזה כל הזמן: שני builds בו-זמניים שכותבים לאותו dataset מסודרים בטור על-ידי טרנזקציות dataset, ולעולם לא תראו פלט קרוע. המאגר הזה הוא איך שנראה להיות האדם שבונה את המגן — במקום לעמוד מאחוריו.

3. המערכת במבט-על

`docker-compose.yml` מגדיר שישה קונטיינרים. שניים הם תשתית (`redis`, `postgres`), אחד הוא צעד הכנה חד-פעמי (`migrate`), ושלושה מריצים את קוד המאגר (`api`, `worker`, `ticker`) — שלושתם נבנים מאותו `image` ונבדלים רק בפקודה שהם מריצים.

- **postgres** — בסיס הנתונים. הבעלים של כל מצב המשימות. המקום היחיד שבו סטטוס של משימה הוא סמכותי.
- **redis** — מחזיק את שלושת `streams` התור, את ה-`sorted set` של המשימות המושהות, ואת הנהלת-החשבונות של מסירת הודעות. אותות שיגור (`dispatch`) בלבד — לעולם לא האמת.
- **migrate** — מריץ `alembic upgrade head` ליצירת/שדרוג הסכימה, ואז יוצא. כל השאר מחכים לו.
- **api** — שרת `FastAPI` (`app/main.py`, ראוטים ב-`app/api/routes.py`). מקבל הגשות, עונה לשאילתות סטטוס, מטפל בביטול/ריצה-מחדש, ומגיב `/health` ו-`/stats`.
- **worker** — מבצע המשימות (`app/worker/runner.py`). לולאה אינסופית: משוך מזהה משימה מהתור, תפוס בעלות (`claim`), הרץ את ה-`handler`, רשום את התוצאה, אשר (`ack`). מרחיבים עם: `docker compose up --scale worker=N`.
- **ticker** — השעון של המערכת (`app/ticker/runner.py`), **עוֹתֵק אֶחָד בְּדִיוֹק**. כל שנייה הוא מקדם משימות מתוזמנות שהגיע זמןן; מדי פעם הוא גם מחלץ משימות שה-`worker` שלהן מת, ומתקן משימות שמעולם לא הגיעו ל-`Redis`.

מקבילה ל-FOUNDRY

המיפוי ל-`Foundry`, שירות-שירות: `api` = נקודת הקצה של שירות ה-`Build` שאתם פוגעים בה כשלווחצים `Build`; `postgres` = בסיס הנתונים של היסטוריית ה-`builds` שמאחורי דף ה-`builds`; `redis streams` = תור ה-`builds`; `worker = executor` = מודול `Spark` שאוסף `builds` מהתור; `ticker` = שירות ה-`schedules` ועוד השרת שמזהה `builds` מתים. `docker compose up --scale worker=5` = הוספת `executors` לאשכול החישוב.

כלל העיצוב האחד שמסביר את כל השאר

פוסטגרס הוא מקור האמת; `Redis` הוא רק שליח מהיר. כל שינוי מצב מקובע (`commit`) בפוסטגרס **קודם**; הודעת ה-`Redis` רק אומרת "משימה `<uuid>` אולי מוכנה, לך תבדוק בבסיס הנתונים". הודעות לא נושאות `payload` — רק את מזהה המשימה. המסקנה: `Redis` יכול לאבד הודעות, למסור אותן פעמיים, או להימחק כליל — ושום מצב משימה לעולם לא יהיה שגוי; במקרה הגרוע משימה מחכה עד שלולאת תיקון שולחת מחדש את ההודעה שלה. החזיקו את הכלל הזה בראש לאורך כל ההמשך; הוא הסיבה שכל מנגנון הוא בטוח.

4. חייה של משימה: המסלול התקין, מקצה לקצה

נעקוב אחרי משימה אחת מהגשה ועד השלמה, עם הקוד האמיתי. לקוח מגיש:

```
POST /jobs
{
  "type": "email",
  "payload": {"to": "user@example.com", "subject": "hi"},
  "priority": "high"
}
```

שלב 1 — ה-API מאמת ורושם (app/api/routes.py, הפונקציה submit_job)

FastAPI מפענח את הגוף לסכימת JobSubmission. אחר כך validate_payload() בודק את ה-payload מול סכימת ה-Pydantic הרשומה **לסוג המשימה הזה** (payload — app/schemas/payloads.py של email בלי שדה 'to' נדחה עם HTTP 422 לפני שכלום נשמר. ואז _create_and_handoff() מכניס שורה לטבלת jobs עם 'status='pending' ומקבע (COMMIT). המשימה עכשיו קיימת באופן עמיד (durable). שום דבר עוד לא נכנס לתור.

שלב 2 — ה-API מכניס לתור את המצביע

enqueue() (app/queue/producer.py) מריץ XADD על ה-stream שמתאים לעדיפות המשימה — כאן jobs:stream:high — עם שדה יחיד: ה-UUID של המשימה. אחר כך mark_synced() מדליק בשורה את הדגל is_synced_to_redis. הדגל קיים מסיבה עדינה שמכוסה בחלק 7.6: אם ה-API מת בין ה-commit ל-XADD, השורה נותרת תקועה עם synced=false, וה-ticker יבחין בכך וישלח אותה מחדש. הלקוח כבר קיבל HTTP 202 עם מזהה המשימה.

שלב 3 — worker מושך את ההודעה (app/queue/consumer.py, הפונקציה read_priority)

כל worker מסתובב בלולאה על read_priority(): בודק את jobs:stream:high, אחר כך normal, אחר כך low, ללא חסימה (non-blocking), וקח את ההודעה הראשונה שנמצאה — כך עבודה בעדיפות גבוהה תמיד מתרוקנת קודם. רק אם שלושתם ריקים הוא צולל לקריאה חוסמת אחת על שלושתם יחד (block_ms=5000), שמחנה את ה-worker בזול עד שמשוּה מגיע. הקריאה משתמשת ב-XREADGROUP — גרסת ה-consumer group שמוסברת בחלק 6 — שמבטיחה שכל הודעה נמסרת לבדיוק worker אחד.

שלב 4 — ה-worker תופס בעלות על המשימה בפוסטגרס (app/worker/runner.py, הפונקציה process_job)

```
if not repo.claim_job(session, job_id):
    log.info("job.skipped", reason="not_claimable")
    return Outcome(ack=True, recycle=False, label="skipped")
```

claim_job() הוא העדכון המותנה האטומי מחלק 2: קבע 'status='processing' ו-started_at=now WHERE id=? AND status IN ('pending','scheduled'). אם הוא מדווח 0 שורות שהשתנו — מישהו אחר כבר תפס את המשימה (או שהיא בוטלה) — ה-worker מאשר (ack) את ההודעה וממשיך הלאה בלי להריץ דבר. זו שכבת סינון-הכפילויות שהופכת מסירת at-least-once לביצוע at-most-once.

שלב 5 — ה-handler רץ, בתוך timeout

`run_handler()` מאתר את הפונקציה הרשומה עבור 'email' ב-`app/jobs/registry.py` וקורא לה — עטופה ב-`run_with_timeout()` (`app/worker/timeout.py`), שמריץ אותה על `thread` טרי ונותן לה `job_handler_timeout_s` (45 שניות) לסיים. במסלול התקין: ה-`handler` ישן את 1-3 השניות המדומות שלו ומחזיר `{"message_id": "..."}.`

שלב 6 — ההשלמה מקובעת, ורק אז ההודעה מאושרת

```
won = repo.complete_job(session, job.id, result, ...) # guarded UPDATE + COMMIT
...
return Outcome(ack=True, ...) # the caller then runs XACK
```

`complete_job()` הוא עוד עדכון מוגן (רק שורת 'processing' יכולה להפוך ל-'completed'), ששומר את ה-`result` כ-JSON ואת `completed_at`. רק אחרי ה-`commit` הזה ה-`worker` מריץ `XACK` על ההודעה — ומודיע ל-`Redis` "המסירה הושלמה, הפסק לעקוב". הסדר מכוון ונושא-משקל: אם ה-`worker` היה מת בין ה-`commit` ל-`ack`, ההודעה הייתה נמסרת שוב בהמשך, ה-`claim_job()` של ה-`worker` הבא היה מוצא את השורה כבר במצב סופי, פשוט מאשר ומדלג. לא מזיק. הסדר ההפוך (`ack` ואז קריסה לפני `commit`) היה מאבד את התוצאה לנצח, בלי מסירה-מחדש שתציל. בספק? קודם הכתיבה העמידה, אחר-כך אות הניקיון.

שלב 7 — הלקוח מתשאל

```
GET /jobs/{id}
-> {"status": "completed", "result": {"message_id": "..."},
   "attempts": 1, "created_at": ..., "started_at": ..., "completed_at": ...}
```

מקבילה ל-**FOUNDRY**

הקשת כולה משקפת `build` ב-`Foundry` בדיוק: הגשה (כפתור `Build`) ← נרשם (`QUEUED` בדף ה-`builds`) ← נאסף על-ידי `executor` (`RUNNING`) ← התוצאה מקובעת (`SUCCEEDED`, עם ארטיפקטים ותזמונים). אפילו חוויית התשואל — רענון דף ה-`build` — היא אותה אינטראקציה ש-`GET /jobs/{id}` מספק.

5. מודל הנתונים: טבלה אחת שמספרת את כל הסיפור

יש טבלת ליבה אחת, `jobs`, המוגדרת על-ידי מחלקת `job` ב-`app/models/job.py`. קריאה שלה עמודה-עמודה היא הזרקה המהירה ביותר להבין את המערכת, כי כמעט כל מנגנון בחלק 7 קיים כדי לנהל נכון עמודה אחת או שתיים מכאן.

- **id** (UUID, מפתח ראשי) — מזהה ייחודי אקראי, נוצר בהכנסה. UUID (ולא 1,2,3...) כדי שמזהים לא יהיו ניתנים לניחוש ויוכלו להיווצר ללא תיאום.
- **type** — איזה handler מריץ את המשימה: `email`, `webhook`, `report` או `batch`.
- **payload** (JSONB) — הקלט המאומת של המשימה, שמור כ-JSON בינארי שאילתתי.
- **status** — לב המודל: `pending`, `scheduled`, `processing`, `completed`, `failed` או `cancelled`. מאונדקס, כי workers ונקודות ה-`list` מסננים עליו בלי הרף.
- **priority** — `high`, `normal` או `low`. קובע לאיזה stream ב-Redis תישלח ההודעה.
- **result / error** (JSONB) — תוצאת הניסיון האחרון: `result` בהצלחה, מידע שגיאה מובנה (סוג חריגה + הודעה) בכישלון.
- **attempts / max_attempts** — הנהלת חשבונות של `retries`: כמה פעמים המשימה נוסתה, והתקרה (ברירת מחדל 4) שאחריה היא נכשלת לצמיתות.
- **created_at / started_at / completed_at** — חותמות זמן של מחזור החיים.
- **scheduled_at** — מתי משימה עתידית אמורה לרוץ; משמש שוב גם כזמן היעד של "retry אחרי backoff".
- **is_synced_to_redis** — דגל תיקון: "האם ההודעה של המשימה באמת הונחה ב-Redis?" (חלק 7.6).
- **progress** — אחוז השלמה, בשימוש ה-`handler` של `batch`.
- **cancel_requested_at** — נקבע על-ידי נקודת הביטול עבור משימה רצה; ה-`handler` של `batch` בודק אותו (חלק 7.7).
- **idempotency_key / idempotency_hash** — הגנה מהגשה כפולה (חלק 7.5). **אינדקס ייחודי חלקי** (partial unique index) אוכף ייחודיות מפתח ברמת בסיס הנתונים — שתי הכנסות בו-זמניות עם אותו מפתח פיזית לא יכולות להצליח שתיהן.

מחזור החיים של הסטטוס

```
(scheduled_at in future)
submit -----> SCHEDULED
|                                     | ticker promotes when due
v                                     v
PENDING <-----+
| worker claims (atomic UPDATE)
v
PROCESSING ----> COMPLETED (result stored)
|               \---> FAILED (after max_attempts) --manual retry--> PENDING
|               \--> back to PENDING/SCHEDULED (retry with backoff)
v
CANCELLED (from pending/scheduled anytime;
           from processing only if the handler cooperates)
```

כל מעבר בדיאגרמה מבוצע על-ידי UPDATE מוגן ב-`app/repository.py` — `fail_job`, `complete_job`, `claim_job` — `retry_to_scheduled`, `retry_to_pending`, `cancel_pending_or_scheduled`, `cancel_job`, `reset_failed_to_pending`. כל אחד מחזיר `True/False` עבור "האם ניצחתי במעבר", והקוראים מסתעפים לפי זה. אין בשום מקום נתיב קוד

שקובע סטטוס בלי להצהיר איזה סטטוס קודם הוא מצפה למצוא — המשמעת הזו היא מה שהופך שחקנים מקביליים לבטוחים.

מקבילה ל-FOUNDRY

מחזור הסטטוסים ממופה אחד-לאחד למצבי build ב-Foundry: *SCHEDULED* = תזמון שטרם נורה; *PENDING* = *QUEUED*; *CANCELLED* = *CANCELED*; *FAILED* = *FAILED*; *COMPLETED* = *SUCCEEDED*; *PROCESSING* = *RUNNING*. טבלת ה-jobs היא דף ה-builds בתור טבלה.

6. צד ה-Redis: Streams, Consumer Groups וקבוצת ההשהיה

כל מה ש-Redis הוא מצב שיגור נגזר ובר-השלכה. קיימים חמישה מפתחות: שלושה streams (jobs:stream:high / :normal / :low), קבוצת צרכנים (consumer group) אחת בשם 'workers' שמשתרעת עליהם, ו-sorted set אחד (jobs:delayed).

6.1 Streams ו-XADD

Stream הוא יומן בהוספה-בלבד. enqueue() מריץ XADD, שמוסיף רשומה שהשדה היחיד שלה הוא job_id=<uuid>. רשומות מקבלות מזהים עולים שמוקצים ע"י השרת, כך ש-stream מסודר מטבעו — בתוך stream של עדיפות אחת, משימות משוגרות בערך לפי סדר הגעה (FIFO).

6.2 Consumer groups: איך N workers חולקים stream אחד בלי לדרוך זה על זה

אם כל worker פשוט היה קורא את ה-stream, כולם היו רואים כל הודעה. **CONSUMER GROUP** פותר את זה: זהו סמן (cursor) בעל-שם על ה-stream, משותף לכל החברים. כש-worker קורא XREADGROUP ומבקש הודעה חדשה אחת, Redis מקדם את הסמן המשותף ומוסר את ההודעה **לאותו worker בלבד** — אף חבר אחר בקבוצה לא יקבל אותה. כל worker מזדהה בשם צרכן ייחודי (hostname + סימנת אקראית, נבנה ב-app/queue/consumer.py).

קריטי להבין: מסירה אינה מחיקה. ברגע שהודעה נמסרת, Redis רושם אותה ב-**PENDING ENTRIES LIST (PEL)** של הקבוצה: "הודעה 17 נמסרה ל-worker_abc ב-12:00:03 וטרם אושרה". ההודעה נשארת שם עד שה-worker קורא XACK. ה-PEL הוא התשובה המובנית של Redis לשאלה "מה היה באוויר כשמשוהו מת?" — וחלק 7.2 מראה את ה-ticker כורה אותו כדי לחלץ משימות יתומות. XAUTOCLAIM היא הפקודה המשלימה: "תן לי כל רשומת PEL שבטלה יותר מ-N מילישניות" — כלומר מסירות שה-worker שלחן השתתק.

מקבילה ל-FOUNDRY

ה-PEL הוא פנקס ה"builds שרצים כרגע" של Foundry. אם executor מת באמצע build, Foundry לא שוכח שה-build היא קיים — הוא יודע בדיוק אילו builds הושאלו ומשגר אותם מחדש. XREADGROUP = executor אוסף build מהתור; XACK = ה-build דיווח מצב סופי; XAUTOCLAIM = השרת שאוסף builds שה-executor שלהם הפסיק לדווח.

6.3 כמה שלושה streams ולא אחד

ל-streams ב-Redis אין מושג של עדיפות — stream הוא בסדר-הוספה קשוח. במקום להילחם בזה, העיצוב הופך את העדיפות להחלטת ניתוב: שלושה streams מקבילים, אחד לכל רמה, שנבחרים פעם אחת בזמן ההכנסה לתור על-ידי settings.stream_for_priority(). ה-workers כופים עדיפות בזמן **קריאה**: בדיקה low ← normal ← high (חלק 4, שלב 3). האלטרנטיבה — sorted set אחד עם ניקוד לפי עדיפות — נדחתה כי pop מ-ZSET לא עוקב אחרי מסירות: אין PEL, אין ack, אין השבה-לפי-בטלה, כך שכל סיפור ההתאוששות-מקריסה של 7.2 היה צריך להיבנות ידנית. DECISIONS.md סעיף 3 מתעד את הפשרה.

6.4 ה-sorted set של ההשהיה

Streams לא יכולים לעכב הודעה עד זמן עתידי. לכן עבודה עתידית חיה במקום אחר: jobs:delayed, sorted set שבו כל איבר הוא UUID של משימה והניקוד שלו הוא חותמת הזמן שבה היא נהיית "בשלה" (due). שני דברים מכניסים לשם משימות: הגשות עם scheduled_at עתידי, ו-retries שממתינים את השהיית backoff שלהם. ה-ticker בודק כל שנייה: "איברים עם ניקוד ≥ עכשיו" ← מקדם כל אחד ל-stream העדיפות שלו (חלק 7.4). sorted set הופך את בדיקת-הבשלות לשאילתת טווח אחת וזולה.

`jobs:deLayed` הוא שירות ה-`schedules`: "ה-`build` הזה צריך לירות ב-02:00" יושב דומם עד שתקתוק השעון של המתזמן מקדם אותו לתור ה-`builds`. `retries` עם `backoff` עושים שימוש חוזר באותו מנגנון — בדיוק כמו ש-`Foundry` מתזמן ריצה-חוזרת אוטומטית של `build` שנכשל כמה דקות קדימה.

7. הבעיות הקשות, אחת-אחת

החלק הזה הוא לב המאגר — ולב המבחן שעבורו הוא נבנה. כל סעיף מציג תרחיש כשל **שיקרה** בכל מערכת מבוצרת, ואז עובר על התשובה של הקוד. הדפוס החוזר: Redis מספק מהירות ואיתות, פוסטגרס מספק את המעבר-המוגן שהופך את הנכונות לבלתי-מתפשרת.

7.1 איסוף כפול: שני workers, משימה אחת

האיום: אותה משימה רצה פעמיים — שני אימיילים נשלחים, דוח מופק פעמיים. מקורות הכפילות: מסירה-מחדש מה-stream אחרי קריסה (מובנה ב-at-least-once), ה-reaper שמכניס מחדש משימה של worker שהיה רק איטי, אדם שלוחץ retry פעמיים.

ההגנה, בשתי שכבות. שכבה 1, Redis: בתוך ה-consumer group, XREADGROUP מוסר כל הודעה לבדיוק צרכן אחד — workers מתחרים אפילו לא רואים את אותה הודעה במקרה הרגיל. שכבה 2, פוסטגרס — זו שבאמת מבטיחה נכונות: העדכון האטומי של `claim_job()` `UPDATE ... WHERE status IN ('pending','scheduled')` לא משנה כמה הודעות כפולות קיימות למשימה J — שתיים, חמש — כל worker שמקבל אחת מריץ את אותה פקודה, ובסיס הנתונים מרשה לבדיוק אחד מהם להעביר את השורה ל-'processing'. כל מפסיד מקבל rowcount 0, מאשר את ההודעה המיותרת שלו, והולך. הודעות כפולות — נסבלות; ביצוע כפול — בלתי אפשרי.

שימו לב מה **אין** כאן: אין בפייתון בדיקת `if job.status == pending`. הצורה הזו של check-then-act היא ה-race condition מחלק 2.5. התנאי חי בתוך ה-WHERE של ה-UPDATE, שם בסיס הנתונים מעריך אותו אטומית יחד עם הכתיבה. מראיינים וסוקרי קוד מחפשים בדיוק את זה.

7.2 התאוששות מקריסת worker: המשימה שנתקעה ב-PROCESSING

האיום: worker תופס משימה, קובע `'status=processing'`, ואז הקונטיינר נהרג ב-OOM. ה-worker לא יכול לנקות אחריו — הוא איננו. בלי התאוששות, המשימה יושבת ב-'processing' לנצח והלקוח מחכה לנצח.

העיצוב מבחין בין שני סוגים שונים של "תקוע", המטופלים בשתי שכבות:

- **handler תקוע, התהליך חי:** כל handler רץ על thread חד-פעמי עם תקציב של 45 שניות (`run_with_timeout`, `app/worker/timeout.py`). ב-timeout ה-worker עצמו מנתב את המשימה לנתיב ה-retry מיידית — בלי עזרה חיצונית. אבל פייתון לא יכול להרוג thread בכוח, כך שה-thread הנטוש אולי עדיין רץ. לכן ה-worker **ממחזר את עצמו**: יוצא עם קוד 1 אחרי `max_handler_timeouts_before_recycle` (ברירת מחדל: כבר ב-timeout הראשון), ו-`restart: on-failure` של docker-compose מרים מחליף טרי ונקי. דרך גסה במכוון אבל אטומה למנוע דליפת משאבים מ-threads זומבים.
- **תהליך מת:** שום דבר בתוך התהליך לא יכול להגיב, ולכן ההתאוששות חיצונית. ההודעה שנתפסה עדיין יושבת ב-PEL, לא מאושרת, וזמן-הבטלה שלה גדל. `reap_stale()` של ה-ticker רץ כל 30 שניות וקורא XAUTOCLAIM עם `visibility_timeout_s = min_idle` (60 שניות): "תן לי כל מסירה שבטלה מעל דקה". עבור כל מזהה משימה שהושב הוא בודק את פוסטגרס ופועל (ראו `_reap_one`, `app/ticker/runner.py`): סטטוס 'processing' ← ה-worker מת באמצע ריצה, נתב דרך נתיב ה-retry הסטנדרטי (7.3); סטטוס סופי ← הודעת רפאים, רק ack; סטטוס לא-מסונכרן ← הקריסה קרתה לפני המסירה ל-Redis, השלם את המסירה במקום (inline). כל ענף מסתיים ב-ack להודעה שהושבה, כך שה-PEL נשאר נקי.

אינווריאנט שקט קושר את השכבות: ולידטור ה-Settings (`app/core/config.py`) **מסרב לעלות** אם `job_handler_timeout_s ≥ visibility_timeout_s`. ה-timeout הפנים-תהליכי (45 ש') תמיד יורה לפני סף ה-reaper (60 ש'), ולכן ה-reaper לעולם לא יכול להשיב משימה מ-worker חי שעדיין בתוך תקציב ה-handler שלו — ה-reaper רואה רק את המתים באמת. לקודד הנחת-סדר כוולידציה בזמן-עלייה, במקום כהערה — דפוס ששווה לגנוב.

ומה עם חיוביים-כוזבים — worker חי שאיכשהו חצה 60 שניות של שתיקה, הושב על-ידי ה-reaper, ואז סיים בכל זאת? גם אז שחיתות נתונים בלתי אפשרית: `complete_job()` מוגן ("רק שורת processing רשאית להשלים"), כך שהצד שמקבע ראשון זוכה בשורה, והכתיבה של המפסיד לא משנה דבר. ה-worker מתעד `job.complete_lost_to_reaper` ומסרב לאשר. התוצאה הגרועה ביותר האפשרית: ריצה אחת עודפת של ה-handler — ולכן handlers במערכות at-least-once צריכים תמיד להיכתב כסובלי-ריצה-חוזרת.

מקבילה ל-Foundry

המקבילה: executor נעצר באמצע build; ה-build יושב רגע ב-RUNNING; הפלטפורמה מבחינה שה-executor הפסיק לשלוח heartbeat ומחזירה את ה-build לתור; אתם רואים ניסיון חדש מופיע. אותו איום, אותה צורת תשובה. הבדל כן אחד, מתועד ב-DECISIONS.md סעיף 5: Foundry משתמש ב-heartbeats חיים ("ה-executor דיווח לפני 4 שניות"), בעוד המאגר הזה משתמש ב-timeout בטלה קבוע — פשוט יותר, אבל הגילוי איטי יותר וחד-מידתי. המחבר מציין heartbeats כדבר הראשון שהיה מוסיף עם עוד זמן.

7.3 Retries עם backoff: כישולן הוא קלט, לא חריגה

משימת ה-webhook נכשלת 20% מהזמן בכוונה, כי כשל חולף הוא נורמלי: שירותים חיצוניים מהבהבים, רשתות מפילות. להפציץ שירות מתקשה ב-retries מידיים רק מחמיר, ולכן retries **ממתנים**, וההמתנות גדלות — "backoff".

פונקציה אחת מחזיקה את כל המדיניות: `schedule_retry_or_fail()` ב-`app/retry.py` — קראו אותה, 40 שורות. הלוגיקה: יהי $n = \text{הניסיון שזה עתה נכשל}$ אם $n \geq \text{max_attempts}$ (4): UPDATE מוגן ל-'failed', שמירת שגיאת ה-JSON, סוף לצמיתות. אחרת חפש את ההמתנה ב-`retry_backoff_schedule = [0, 30, 120]`: אחרי הכישולן הראשון — retry מידי (UPDATE מוגן חזרה ל-'pending' + XADD עכשיו); אחרי השני — 30 שניות; אחרי השלישי — 120 שניות — במקרים המושהים המשימה עוברת ל-'scheduled' ונכנסת ל-sorted set `jobs:delayed`, וה-ticker מקדם אותה כשמגיע זמנה. משימה חסרת-מזל מקסימלית חיה כך: ניסיון, כישולן, ניסיון, כישולן, המתנה 30 ש', ניסיון, כישולן, המתנה 120 ש', ניסיון, כישולן ← FAILED עם השגיאה האחרונה שמורה. אדם יכול אז לבדוק אותה ולהריץ `POST /jobs/{id}/retry`, שמאפס אותה ל-pending עם תקציב ניסיונות רענן.

החלק האלגנטי: **מי** קורא לפונקציה הזו. worker שה-handler שלו זרק חריגה קורא לה סינכרונית — כך ה-retry הראשון מידי" בכלל אפשרי. worker שה-handler שלו עבר timeout קורא לה. וה-reaper של ה-ticker קורא לה עבור משימות שה-worker שלהן מת בלי להספיק לקרוא לכלום. חריגת handler, תקיעה ומוות-תהליך מתנקזים כולם לנתיב קוד אחד, כך שספירת-ניסיונות וסמנטיקת כישולן-סופי לא יכולות להיסחף זו מזו בין מצבי כשל. כשאתם מוצאים את עצמכם בונים נתיב התאוששות שני, מעט-שונה — עצרו ונתבו אותו דרך הראשון.

מקבילה ל-Foundry

builds ב-Foundry מנסים שוב אוטומטית כשלי תשתית חולפים עם השהיות גדלות, ואז מוותרים ומסמנים FAILED בשביל בן-אדם. נקודת ה-retry הידנית היא כפתור "Rerun" על build שנכשל.

7.4 משימות מתוזמנות ותקתוק השעון של ה-ticker

הגשה עם `scheduled_at` עתידי ← ה-API שומר את השורה כ-'scheduled' ומוסיף את המזהה ל-`jobs:delayed` עם ניקוד זמן-היעד — הוא לא נוגע ב-stream בכלל. כל שנייה, `promote_due()` של ה-ticker שואל את ה-sorted set על איברים בשלים. לכל אצווה הוא: שולף עדיפויות מפוסטגרס (ומדלג על מזהים שהשורה שלהם בוטלה או נמחקה — הם מושמטים, לא נכנסים לתור), מריץ XADD לכל שורד אל stream העדיפות שלו ו-ZREM מהקבוצה בסיבוב-רשת אחד עם pipeline (`app/queue/delayed.py`), ואז מעביר את השורות ל-pending ב-UPDATE אחד מרוכז.

דקות ביצועים אחת: אצוות מוגבלות ל-`ticker_batch_size` (100), אבל אם תקתוק קידם אצווה **מלאה**, הלולאה מדלגת על שנת-השנייה שלה ומרוקנת שוב מיד. בלי זה, 10,000 משימות שבשלות כולן ב-09:00 היו מטפטפות על

פני 100 שניות של פיגור מלאכותי. עם זה, ה-ticker טוחן את הצבר במהירות ש-Redis ופוסטגרס מרשים. חפשו את צורת ה"רוקן-עד-שלא-מלא" בכל פעם שסוקר בקצב-קבוע פוגש קלט פרצי.

מקבילה ל-Foundry

זהו המתזמן של Foundry יורה תזמון מבוסס-זמן: *schedules* בשלים הופכים ל-*builds* בתור. דאגת פרץ-הריקון היא "גל ה-2 בלילה" — מאות *schedules* שיוזמים באותה דקה אסור שיסתדרו בטור מאחורי מכסה מלאכותית של פעם-בתקתוק.

7.5 אידמפוטנטיות: אותה הגשה, פעמיים

האיום: לקוח מגיש, התשובה אובדת בשיהוק רשת, הלקוח (או ספריית ה-retry שלו) מגיש שוב. בלי הגנה: שתי משימות זהות, שני אימיילים. הלקוח לא יכול להבדיל בין "הבקשה שלי אבדה" ל"התשובה שלי אבדה" — רק השרת יכול לסנן כפילויות.

החזרה: הלקוח שולח `idempotency_key` — מחרוזת כלשהי לבחירתו, למשל 'welcome-email-user-4711'. ב-submit_job (app/api/routes.py): מחושב hash קנוני של `type+payload` (app/idempotency.py) — "קנוני" = ה-JSON מסודר לפי מפתחות לפני הגיבוב, כך שסדר מפתחות לא משנה את ה-hash; אם משימה עם המפתח כבר קיימת ו-**ה-hash תואם** — מוחזרת המשימה הקיימת עם HTTP 200, הכפילות לא יוצרת דבר; אם המפתח קיים עם **hash שונה** — HTTP 409: הלקוח ממחרז מפתח לעבודה שונה, תמיד באג-לקוח שראוי לדחייה רועשת.

המרוץ: שתי הגשות זהות מגיעות **בו-זמנית**, שתיהן מחפשות את המפתח, שתיהן לא מוצאות, שתיהן מכניסות. קוד אפליקציה לא יכול לסגור את הפער הזה — רק בסיס הנתונים יכול. טבלת jobs נושאת אינדקס ייחודי חלקי על `idempotency_key` (חלק 5), כך ש-INSERT אחד מצליח והשני זורק IntegrityError; הראוט תופס, מבצע rollback, קורא-מחדש את השורה שהמנצח יצר, ומחזיר אותה. שני הלקוחות מקבלים את אותו מזהה משימה. הלכח הכללי חוזר על 7.1: ייחודיות תחת מקבילות נאכפת באילוצים בבסיס הנתונים, לעולם לא ב-look-before-you-insert בקוד.

מקבילה ל-Foundry

רעיון קרוב ב-Foundry: הרצה-מחדש של build על קליטים שלא השתנו לא מכפילה נתוני פלט — טרנזקציות dataset גורמות לפעולה להתכנס. ה-409 על אי-התאמת hash מזכיר את Foundry שמסרב לסמנטיקה סותרת על אותה זהות במקום לנחש.

7.6 הדגל is_synced_to_redis: תיקון התפר בין שתי מערכות

ה-API כותב לשתי מערכות ברצף: COMMIT בפוסטגרס, ואז XADD ב-Redis. עובדה 2 מחלק 2.5 אומרת שהתהליך יכול למות בדיוק ביניהן — ולהשאיר משימה אמיתית ומקובעת שאף worker לעולם לא ישמע עליה, כי ההודעה שלה מעולם לא נשלחה. את תפר ה"כתיבה הכפולה" הזה אי אפשר לסגור בטרנזקציה, כי אין טרנזקציה שמשתרעת על פוסטגרס וגם על Redis. לכן במקום **למנוע** את הפער, העיצוב **מגלה ומתקן** אותו: כל שורה מתחילה עם `is_synced_to_redis=false` ומועברת ל-true רק **אחרי** שה-XADD שלה הצליח. `reconcile_orphans()` של ה-ticker רץ כל 60 שניות: כל שורה לא-סופית שעדיין לא מסונכרנת אחרי תקופת חסד של 10 שניות מקבלת שליחה-מחדש של ההודעה שלה (ל-stream או לקבוצת ההשהיה, לפי הסטטוס) ומסומנת מסונכרנת. במקרה הגרוע, משימה שההכנסה-לתור שלה נקטעה מתחילה באיחור של כדקה. זה "ריפוי-עצמי במקום מניעה" — וזה גם המנגנון שהופך את Redis לבר-החלפה לגמרי: מחקו את Redis, והרקונסיליאציה תבנה מחדש את התור מפוסטגרס.

7.7 ביטול: קל — עד שהמשימה רצה

ביטול משימת pending או scheduled הוא UPDATE מוגן ככל אחר — אם ניצח, השורה 'cancelled', וכל הודעה מיושנת עבורה תמות מאוחר יותר במחסום ה-claim. (המזהה גם מוסר מקבוצת ההשהיה ב-ZREM — no-op לא-מזיק אם איננו שם.) ביטול משימת **PROCESSING** שונה מהותית: ה-handler מתבצע על worker אי-שם, ואי אפשר להרוג thread של פייתון בבטחה מבחוץ. לכן ביטול תוך-כדי-ריצה הוא **שיתופי (cooperative)**: נקודת

הביטול רק קובעת `cancel_requested_at` ומחזירה HTTP 202 — "התבקש, לא מובטח". ה-`handler` חייב מרצונו לבדוק את הדגל ולנטוש; רק ה-`handler` של `batch` עושה זאת (בין פריטים, דרך אובייקט ה-`context` שלו), זורק `JobCancelled`, וה-`worker` רושם תוצאת ביטול עם סיכום חלקי. משימות `email/webhook/report` אורכות שניות, כך שלא היה שווה להוסיף להן `polling` — פשרה מתועדת (`DECISIONS.md` סעיף 4) וריאליסטית: ביטול שיתופי הוא איך ש-`Spark`, `graceful shutdown` של `Kubernetes` ורוב מערכות המשימות באמת מתנהגים.

ראוט הביטול עצמו (`app/api/routes.py`) הוא שיעור קטן במקביליות: המשימה עשויה באופן חוקי להתנדנד בין `pending` ל-`processing` (נחת `retry`) **בזמן** שהבקשה מטופלת, ולכן הראוט מנסה בלולאה עד 3 פעמים את המעברים המוגנים שלו — נסה ביטול-כ-ממתינה, אחרת נסה בקשת-ביטול-כ-רצה, אחרת קרא-מחדש והחלט-מחדש — במקום להניח שהסטטוס שנקרא לפני מיקרו-שנייה עדיין תקף. סטטוסים הם מטרות נעות; קוד שקורא אחד ופועל עליו שורה אחר-כך כבר מיושן.

מקבילה ל-**FOUNDRY**

הריגת `build` ב-`RUNNING` היא אותה חוויה: ה-`build` לא נעלם ברגע לחיצת `cancel` — ה-`executor` מבחין באות הביטול בבדיקה הבאה שלו ומסיים בהדרגה, לפעמים גומר קודם את השלב הנוכחי. ואחוז ההתקדמות של `batch` הוא פס ההתקדמות של `build`.

7.8 כיבוי מסודר (Graceful shutdown)

כש-`docker` עוצר קונטיינר הוא שולח `SIGTERM`, ממתין תקופת חסד (50 שניות ל-`workers` — במכוון ארוכה מתקציב ה-45 שניות של ה-`handler`), ואז הורג. מטפל האותות של ה-`worker` (`run_forever`) רק מדליק דגל; הלולאה מסיימת את המשימה הנוכחית עד הסוף — כולל `commit` ו-`ack` — ורק אז בודקת את הדגל ויוצאת. שום עבודה לא ננטשת בדיפלוי או `scale-down` שגרתיים. ואם ההריגה נוחתת בכל זאת? המשימה מחולצת ע"י 7.2, כי התאוששות-מקריסה לא שואלת **למה** ה-`worker` מת. זה הרווח השקט של התאוששות-מקריסה טובה: כיבוי מסודר הופך לאופטימיזציה, לא לדרישת נכונות.

8. סיור מודרך בקוד

מפה של `app/`, מסודרת בערך לפי סדר הקריאה המומלץ. סדר קריאה מוצע לסיבוב ראשון: `models/job.py` → `repository.py` → `api/routes.py` → `queue/` → `worker/runner.py` → `retry.py` → `ticker/runner.py`.
הזה עוקב אחרי הנתונים במקום אחרי רשימת התיקיות.

- `app/main.py` — עליית FastAPI: קונפיגורציה, לוגים, לקוחות DB/Redis, חיבור ראוטים.
- `app/api/routes.py` — כל שש נקודות הקצה: `GET /jobs`, `GET /jobs/{id}`, `POST /jobs` (רשימה מסוננת עם cursor), `GET /stats`, `GET /health`, `POST /jobs/{id}/retry`, `POST /jobs/{id}/cancel`.
- `app/api/deps.py` — הזרקת תלויות: session של DB ולקוח Redis לכל בקשה.
- `app/models/job.py` — טבלת ה-Job (חלק 5). — `app/models/base.py` בסיס SQLAlchemy.
- `app/repository.py` — **הקובץ החשוב ביותר לנכונות:** כל מעבר סטטוס כ-UPDATE מוגן שמחזיר ניצחתי/הפסדתי. אף קובץ אחר לא משנה סטטוס.
- `app/schemas/` — שכבת Pydantic: `enums.py` (סטטוס/סוג/עדיפות), `api.py` (צורות בקשה/תשובה), `payloads.py` (ולידציית payload לכל סוג), `results.py` (צורות תוצאה).
- `app/queue/producer.py` — `XADD` = `enqueue()`. — `app/queue/consumer.py` שמות צרכנים, `ensure_group()`, `ack()`, ו-`read_priority()` עם בדיקת `low` ← `normal` ← `high`. — `app/queue/delayed.py` עוזרי ה-`sorted set`: `schedule()`, `due_job_ids()`, `promote()` עם `pipeline`.
- `app/worker/runner.py` — לולאת ה-`worker` ו-`process_job()` (חלקים 4 / 7.2). — `app/worker/timeout.py` עטיפת ה-`thread` של `run_with_timeout`. — `app/worker/context.py` `PgJobContext`: עוזר פונה-ל-`handler` לעדכוני התקדמות ובדיקות ביטול. — `app/worker/__main__.py` — `python -m app.worker`.
- `app/jobs/handlers.py` — ארבעת ה-`handlers`. — `app/jobs/registry.py` מילון מפורש: סוג ← `handler` (בלי `dispatch` דינמי מקלט משתמש, בכוונה).
- `app/retry.py` — `backoff_delay()` + `schedule_retry_or_fail()` (חלק 7.3).
- `app/ticker/runner.py` — `promote_due()`, `reconcile_orphans()`, `reap_stale()` ולולאת התקתוק (7.2/7.4/7.6).
- `app/idempotency.py` — `canonical_hash()`. — `app/cursor.py` קידוד/פענוח cursor אטום לדפדוף.
- `app/core/config.py` — `Settings` ממשתני סביבה, כולל אינווריאנט ה-`timeout`. / `app/core/db.py` `redis.py` — יצרני לקוחות. — `app/core/logging.py` הגדרת `structlog` ל-JSON.
- `app/observability.py` — בדיקות `health` ואיסוף `stats` (עומקי תורים, ספירות לפי סטטוס).

מוסכמות ששוות תשומת-לב

- כל שורת לוג היא JSON מובנה דרך `structlog`, עם `job_id` / `message_id` / `consumer` קשורים כהקשר — כל חייה של משימה אחת, על-פני שלושה שירותים, הם שאילתת סינון אחת. הדפסות `print` אסורות בכללי הפרויקט.
- מעברים מוגנים מחזירים בוליאנים, והקוראים **תמיד** מסתעפים על 'won' — חפשו (`grep`) את `won` כדי למצוא כל נקודת החלטה מקבילית בקוד.
- קונפיגורציה היא משתני-סביבה דרך `pydantic-settings`; הקונטיינרים מקבלים `DATABASE_URL` ו-`REDIS_URL` ב-`docker-compose.yml` ושום דבר לא כתוב קשיח.

9. בדיקות, הרצה ותצפית

9.1 הרצת המערכת

```
docker compose up --build          # api :8000, 1 worker, ticker, pg, redis
docker compose up --scale worker=4 # more executors

curl -X POST localhost:8000/jobs -H 'Content-Type: application/json' \
  -d '{"type":"webhook","payload":{"url":"https://x.test/hook"}}'
curl localhost:8000/jobs/<id>      # poll status
curl localhost:8000/health        # dependency checks
curl localhost:8000/stats         # queue depths + per-status counts
```

סדר העלייה מקודד ב-compose: postgres/redis להיות בריאים ← migrate מריץ את הסכימה ← api/worker/ticker עולים. סוג ה-webhook הוא הכיפי להדגמה: הגישו עשרה וצפו בכ-שניים מהם עוברים retry עם backoff בלוגי ה-worker (docker compose logs -f worker).

9.2 חבילת הבדיקות

מריצים הכל עם: `uv run pytest`. (uv הוא מנהל החבילות/סביבות של הפרויקט — הכלל הוא `uv run <cmd>`, לעולם לא `pip/venv` גולמיים.) הבדיקות מפוצלות לשתי שכבות, והפיצול עצמו הוא שיעור:

- `tests/unit/` — לוגיקה טהורה, בלי שירותים: אריתמטיקת backoff, עטיפת ה-timeout, ולידציית payloads, handlers (עם אקראיות מוזרקת כך ש"20% כישלון" של ה-webhook נכפה דטרמיניסטית), אינווריאנטים של קונפיגורציה, קידוד cursor, גיבוב אידמפוטנטיות.
- `tests/integration/` — רצות מול פוסטגרס ו-Redis **אמיתיים** מה-compose: איסוף דרך ה-consumer group, זרימת ה-retry המלאה, ה-reaper משיב הודעה לא-מאושרת, קידום מושהים, ביטול, הגשה אידמפוטנטית דרך ה-API, התקדמות batch, worker מקצה-לקצה, מיגרציות שנטענות נקי.

למה Redis אמיתי ולא fake? היסטוריית ה-git של המאגר מכילה את התשובה: תלות fakedredis **הוסרה בכוונה**. הערבויות של המערכת חיות בסמנטיקה המדויקת של Redis — הנהלת חשבונות של PEL, כללי הבטלה של XAUTOCLAIM, סמני קבוצות. fake שמקרב את הסמנטיקה עובר בדיקות בדיוק היכן שהאמיתי היה נשבר — גרוע יותר מאשר בלי בדיקה. שימו לב גם **אין** נבדקת התאוששות מקריסה (`tests/integration/test_reaper.py`): אף תהליך לא נהרג — הבדיקה קוראת הודעה, במכוון לא מאשרת לעולם, ואז מפעילה את ה-reaper עם סף בטלה זעיר. לייצר את **המצב** הביניים המדויק שקריסה משאירה — מהיר ודטרמיניסטי יותר מלביים את הקריסה.

מקבילה ל-FOUNDRY

אנלוגיה: בדיקות `unit = checks` על לוגיקת transform עם קלטים מדומים; בדיקות אינטגרציה = הרצת ה-pipeline באמת מול תשתית אמיתית. ו"אל תסמכו על fake של הפלטפורמה שלכם" — זו הסיבה שולידיציית pipelines של Foundry קורית בסוף על Foundry, לא על Spark מקומי שרק דומה לו.

9.3 תצפית על המערכת

עונה "האם אני מגיע לפוסטגרס ול-Redis" (503 אם לא) — מתאים ל-healthchecks של קונטיינרים; ה-compose מחבר את זה). `GET /stats` מדווח ספירות משימות לפי סטטוס פלוס עומקי תורים ב-Redis — המספרים שחשובים תפעולית: pending שעולה כשה-workers פנויים = השיגור שבור; גודל קבוצת-ההשהיה שעולה מעבר לזמני היעד = ה-ticker נפל (הוא עותק-יחיד — הרכיב האחד במערכת שדורש "לצפות בו"); PEL שגדל = workers מתים או תקועים. כל שורת לוג נשלחת כ-JSON עם מזהה המשימה מוצמד, כך ש"grep על ה-job_id משחזר את הביוגרפיה המלאה של כל משימה על-פני api, worker ו-ticker.

10. מגבלות, פשרות ומילון מונחים

10.1 מגבלות כנות (כולן מתועדות ב-DECISIONS.md)

- **עדיפות קשיחה יכולה להרעיב:** שיטפון רציף של משימות high מעכב normal/low ללא הגבלה. פתח מילוט מתועד: הקדשת worker ל-stream הנמוך דרך רשימת ה-streams שלו.
- **התאוששות מבוססת-timeout, לא heartbeat:** משימה של worker מת מחכה עד ~60 שניות לחילוץ, והסף חייב להיות מכיל לסוג המשימה האיטי ביותר. heartbeats הם השיפור המוצהר "עם עוד זמן".
- **ה-ticker הוא עותק יחיד:** הלולאות שלו לא בטוחות לריצה כפולה במקביל (אין בחירת-מנהיג), ולכן אסור להרחיב אותו. עומס העבודה שלו זעיר; החיות שלו ראויה לניטור הצמוד ביותר.
- **מדיניות retry אחת לכל סוגי המשימות:** max_attempts ולוח ה-backoff הם הגדרות גלובליות, לא פר-משימה.
- **ביטול תוך-כדי-ריצה עובד רק ל-batch** (polling שיתופי, 7.7).
- **אין אימות (authentication):** מחוץ לתחולת המבחן; הדבר הראשון להוסיף לפני חשיפה אמיתית.

10.2 מילון מונחים

- **Job (משימה)** — יחידת עבודת-רקע אחת: סוג, payload, מחזור חיים.
- **Queue (תור)** — חוצץ בין "עבודה התבקשה" ל"עבודה בוצעה"; כאן, Redis streams.
- **Producer / Consumer** — הצד שמכניס לתור / הצד שמושך ומבצע.
- **Broker** — תוכנת-ביניים ייעודית לתורים (SQS, Kafka, RabbitMQ); Redis ממלא כאן את התפקיד.
- **Stream** — יומן הוספה-בלבד של Redis; XADD מוסיף, XREADGROUP קורא דרך קבוצה.
- **Consumer group** — סמן משותף בעל-שם, כך ש-N צרכנים מתחלקים ב-stream בלי חפיפה.
- **PEL (Pending Entries List)** — פנקס המסירות-שטרם-אושרו של Redis; שביל פירורי-הלחם של ההתאוששות.
- **XACK** — "סיימתי, הפסק לעקוב אחרי המסירה". **XAUTOCLAIM** — "תן לי מסירות שבטלות מעל N מילישניות" (כלי ה-reaper).
- **Sorted set (ZSET)** — מבנה Redis ממוין לפי ניקוד; כאן, מזהי משימות לפי זמן-יעד.
- **Transaction (טרנזקציה)** — קבוצת שינויי DB בשיטת הכל-או-כלום.
- **Atomic (אטומי)** — בלתי-ניתן-לחלוקה; אף תהליך אחר לא יכול להשתחל פנימה.
- **Guarded transition (מעבר מוגן)** — UPDATE עם המצב-הקודם-הצפוי בתנאי ה-WHERE; מחזיר ניצחון/הפסד. המהלך החתום של המאגר.
- **Race condition** — באג משזירה ביש-מזל של שחקנים מקביליים, למשל check-then-act.
- **Idempotent (אידימפוטנטי)** — בטוח להחלה פעמיים עם אותה תוצאה כמו פעם אחת.
- **At-least-once** — חוזה מסירה: לעולם לא אובד, אולי מוכפל; משלבים עם סינון-כפילויות ומקבלים exactly-once אפקטיבי.
- **Visibility timeout** — זמן הבטלה שאחריו מסירה לא-מאושרת מוחזקת כנטושה (60 ש' כאן).
- **Backoff** — השהיות גדלות בין retries (0, 30, 120 שניות כאן).
- **Reaper / Reconciler / Promoter** — שלוש לולאות ה-ticker: חילוץ משימות של workers מתים / תיקון משימות שלא הוכנסו לתור / שחרור משימות מתזמנות בשלות.

- **Worker recycling** — worker יוצא אחרי timeout של handler כדי שהאורקסטרטור ירים אותו נקי.
- **Ticker** — שירות המתזמן-פלוס-שרת-הניקיון של המאגר; עותק יחיד.

10.3 לאן ממשיכים מכאן

קראו את DECISIONS.md הבא בתור — עכשיו, כשאתם מכירים את המנגנונים, דיוני הפשרות ינחתו נכון. אחר-כך עשו את התרגיל שמלמד הכי הרבה: הרימו את הסביבה, הגישו משימת batch, הריצו `docker compose kill`, `worker` באמצע הריצה, וצפו בלוגים כשה-ticker מחלץ אותה ו-`worker` טרי משלים את ה-retry. התאוששות שצפיתם בה כמו-עיניכם היא התאוששות שאתם מבינים. אחרי זה, בחרו מגבלה כלשהי מ-10.1 ושרטטו את התיקון — מדיניות retry פר-סוג-משימה היא נקודת הכניסה העדינה ביותר; heartbeats היא המלמדת ביותר.